

THE PROMPT LADDER

RagBucket Case Study Prompt · Iterating One Layer at a Time

Goal

Build a reusable prompt that turns real project information into an honest technical case study without inventing metrics, results, or production claims.

Baseline — Weak Prompt

Prompt

Write a case study for my RagBucket project.

Representative output

Claude did not write the case study. It said it did not know what RagBucket was and asked for raw material across seven areas: what the project does, why I built it, stack, current state, failures, measured numbers, and missing pieces.

What changed? Nothing. This was the original weak prompt.

What improved? Nothing to compare yet. The model correctly avoided inventing project details.

What still failed? The prompt had no context, goal detail, or expected structure, so it produced clarification questions instead of a case study.

What would I try next? Add a clear goal and output structure.

Version 1 — Add Structure

Prompt

Write a short case study for my RagBucket project.

Explain:

1. The problem
2. What I built
3. The result

Representative output

Claude still refused to fabricate details, but the clarification became narrower. It asked for three specific areas: the real problem, the actual stack and workflow, and the real result or evidence.

What changed? I added a clear three-part case-study structure.

What improved? The questions became focused around the three beats I needed.

What still failed? The prompt still contained no actual RagBucket context.

What would I try next? Add the target audience.

Version 2 — Add Audience

Prompt

Write a short case study for my RagBucket project.

Audience: Small startup teams looking for practical open-source AI/ML solutions.

Explain:

1. The problem
2. What I built
3. The result

Output observation

Adding an audience can improve framing, but it still does not supply project facts. Audience alone cannot solve the missing-context problem.

What changed? I defined one specific audience.

What improved? The intended reader and relevance criteria became clearer.

What still failed? The model still lacked the facts needed to write a project-specific case.

What would I try next? Add my voice card.

Version 3 — Add Voice

Prompt layer added

Voice: direct, curious, honest, technical, plain, no buzzwords. Use short, natural sentences. Do not make the project sound more successful or production-ready than it is.

Output observation

The tone becomes more aligned with my portfolio, but voice instructions still cannot replace missing evidence.

What changed? I added my standing voice card.

What improved? The desired tone became explicit and generic portfolio language was discouraged.

What still failed? A well-styled answer could still guess project facts.

What would I try next? Add factual constraints.

Version 4 — Add Constraints

Prompt layer added

Rules: only use facts I provide. Do not invent benchmarks. Do not exaggerate success. Do not claim production readiness. Keep limitations visible.

Output observation

The prompt now protects against the main failure mode I found during the RagBucket and LazyTune interviews: confident claims that the evidence does not support.

What changed? I added explicit factual and claim constraints.

What improved? The model is told exactly what it must not infer or inflate.

What still failed? The prompt still assumes I have already supplied enough project context.

What would I try next? Make the model interview me before drafting.

Version 5 — Add Interview-First Workflow

Prompt layer added

Before writing anything, interview me one question at a time about the real problem, what I built, design decisions, trade-offs, failed ideas, limitations, real results, and what I would do differently.

Output observation

This changes the workflow from 'generate copy' to 'extract evidence first.' It is the approach that exposed the portability reframe in RagBucket and the unsupported speed claim in LazyTune.

What changed? I added a one-question-at-a-time interview before drafting.

What improved? The model can pull out specific decisions and challenge unsupported claims before writing.

What still failed? The process is longer than a one-shot prompt, but that is an acceptable trade-off for project-specific, defensible writing.

What would I try next? Turn the successful layers into one reusable prompt.

Final Reusable Prompt

I want to create a technical case study for one of my AI/ML projects.

Before writing anything, interview me one question at a time until you understand:

- the real problem
- what I built
- important design decisions
- trade-offs
- failed ideas
- limitations
- real results
- what I would do differently

Audience: Small startup teams looking for practical open-source AI/ML solutions.

Voice: Direct, curious, honest, technical, plain, no buzzwords. Use short and natural sentences.

Rules: Do not invent benchmarks. Do not exaggerate success. Do not claim production readiness. Only use facts I provide. Keep limitations visible.

After the interview, write the case study in three beats:

1. The Problem
2. What I Did and Decided
3. What Came of It

End with: **What I'd Do Differently.**

Then list any spot where you had to guess so I can correct it.

Final Reflection

The biggest improvement was not adding more adjectives or a longer role prompt. It was changing the workflow so the model interviews me before it writes. The prompt became useful when it started extracting real decisions and evidence instead of trying to generate a polished story from a project name.